

# **Lectures**

**MATH 107: Introduction to Linear Algebra — Fall 2026**

Daily reading is listed with each meeting. Free Strang/MIT files and the Lachniet Octave PDF are collected on [Resources](#).

Weeks 1–4 outlines are posted. Lectures through Week 4 stay one class day later than the original calendar (Lecture 01 began Wednesday, August 26). **Week 5 order:** dependence → span → [Lab 2](#) (software practice on Lectures 08–10) + Quiz 4. **Week 6:** basis (no quiz) → synthesis/review → Midterm 1 on Friday, October 2. **Quizzes and labs are on Fridays;** midterms too when scheduled.

## Week 1

### Mon Aug 24 — First day

No lecture outline. Class started Wednesday, August 26 with [Lecture 01](#).

### Wed Aug 26 — Why linear algebra? Vectors as representations

[Lecture 01 page](#)

**Reading:** Boyd 1.1

By the end of class, students should be able to:

1. Identify objects that can be represented as vectors.
2. Interpret vector entries, units, and indexing in context.
3. Distinguish scalars from vectors.
4. Translate a small data, position, signal, or measurement problem into vector form.

### Fri Aug 28 — Vector arithmetic and geometry

[Lecture 02 page](#)

**Reading:** Boyd 1.2–1.3

By the end of class, students should be able to:

1. Add, subtract, and scale vectors.
2. Interpret those operations geometrically.
3. Plot vectors in  $\mathbb{R}^2$  and recognize corresponding operations in  $\mathbb{R}^3$ .
4. Check whether an answer has sensible dimensions and units.

## Week 2

### Mon Aug 31 — Linear combinations

[Lecture 03 page](#)

**Reading:** Linear combinations (class notes), [Lab 0](#), and Lachniet Ch. 1 (Getting Started).

**Note:** No prior coding assumed. Lachniet is one PDF on [Resources](#); chapters are listed here.

By the end of class, students should be able to:

1. Form and interpret linear combinations.
2. Explain the meaning of coefficients in context.
3. Enter, index, and modify vectors in Octave or Python.

4. Perform vector arithmetic and create a basic plot.
5. Modify supplied code and explain its output or error message.

## Wed Sep 2 — Inner product

[Lecture 04 page](#)

**Reading:** Boyd 1.4; Boyd 1.1 stack and slice (Quiz 1 prep)

By the end of class, students should be able to:

1. Compute a dot product.
2. Interpret it as a weighted sum.
3. Interpret its sign and magnitude as alignment.
4. Recognize orthogonal vectors from a zero dot product.

## Fri Sep 4 — Linear and affine functions

[Lecture 05 page](#)

**Reading:** Boyd 2.1 and 2.3

**Note:** Boyd's calculus-dependent Taylor-approximation material is optional.

By the end of class, students should be able to:

1. Distinguish linear from affine functions in examples.
2. Test superposition numerically on simple functions.
3. Construct a function  $a^\top x + b$  for a contextual quantity.
4. Evaluate a model by hand and computationally.

## Week 3

### Mon Sep 7 — Labor Day — no class

## Wed Sep 9 — Linear functions in applications

[Lecture 06 page](#)

**Reading:** Applications of linear and affine models

By the end of class, students should be able to:

1. Model a score, cost, prediction, or aggregate quantity.
2. Interpret model coefficients and intercepts.
3. Compare outputs from competing models.
4. State a reasonable limitation or assumption of a linear model.

## Fri Sep 11 — Norm, distance, RMS, and variation

[Lecture 07 page](#)

**Reading:** Boyd 3.1–3.3

By the end of class, students should be able to:

1. Compute vector norms and distances.
2. Interpret norm and distance geometrically and contextually.
3. Relate norm to RMS and, where appropriate, variation around an average.
4. Use software to calculate distances in dimensions too large for hand computation.

## Week 4

### Mon Sep 14 — Angle and similarity

[Lecture 08 page](#)

**Reading:** Boyd 3.4

By the end of class, students should be able to:

1. Compute cosine similarity or an angle between vectors.
2. Distinguish closeness in position from similarity in direction.
3. Choose an appropriate similarity measure for a stated purpose.
4. Explain why normalization may change a comparison.

### Wed Sep 16 — Nearest neighbors

[Lecture 09 page](#)

**Reading:** Boyd §3.2 nearest neighbor; exercises 3.10–3.11, 3.24

**Note:** Possible contexts: species measurements, geographic sites, songs, or sensor readings. (Boyd Ch. 4 begins clustering; NN is Chapter 3.)

By the end of class, students should be able to:

1. Formulate a nearest-neighbor question using vectors.
2. Find a nearest neighbor in a small dataset by hand.
3. Explain how feature scale can affect the answer.
4. Justify a choice of distance or similarity measure.

### Fri Sep 18 — Clustering and $k$ -means

[Lecture 10 page](#)

**Reading:** Boyd 4.1–4.3

**Note:** Same day as Quiz 3 and short Lab 1. Use a small dataset students can visualize.

By the end of class, students should be able to:

1. Explain the goal of clustering.
2. Compute one assignment/update iteration of  $k$ -means.
3. Interpret a centroid as a vector.
4. Recognize that clustering output depends on representation, scaling, and initialization.

## Week 5

### Mon Sep 21 — Linear dependence as redundancy

[Lecture 12 page](#)

**Reading:** Boyd 5.1

By the end of class, students should be able to:

1. Recognize simple dependent and independent collections.
2. Construct a dependence relationship.
3. Interpret dependence as redundant information.
4. Explain why redundancy matters in data, measurement, and computation.

### Wed Sep 23 — Span, independence, and subspaces

[Lecture 13 page](#)

**Reading:** Traditional vocabulary supplement to Boyd

By the end of class, students should be able to:

1. Define span using linear combinations.
2. Determine whether a vector lies in a simple span.
3. Interpret spans geometrically as lines, planes, or higher-dimensional collections.
4. Recognize basic subspaces and distinguish them from shifted lines or planes.

### Fri Sep 25 — Lab 2 (software practice) + Quiz 4

[Lab 2](#) · [Quiz 4](#) · [Syntax reference](#)

**Reading:** [Lab 1](#) skills; Lachniet Ch. 1; Lectures 08–10. No new Boyd section.

**Note:** This is a **lab day**, not a theory lecture — distances / nearest neighbors / optional  $k$ -means assignment in Octave or Python, then Quiz 4 on Week 4 material. Dependence and span from Mon/Wed appear on Midterm 1; basis is Monday of Week 6.

By the end of class, students should be able to:

1. Load and inspect a dataset represented by vectors.
2. Compute many distances with Octave or Python.
3. Adapt supplied code to identify nearest observations or clusters.
4. Validate computational output on a small hand-checkable subset.
5. Interpret results in the original context.

## Week 6

### Mon Sep 28 — Basis, dimension, and orthonormal vectors

[Lecture 14 page](#)

**Reading:** Boyd 5.2–5.3

**Note:** Gram–Schmidt is a important application. No quiz today — Midterm 1 is Friday.

By the end of class, students should be able to:

1. Explain what a basis accomplishes.

2. Identify a basis and dimension in simple examples.
3. Use linear combination, span, subspace, independence, basis, and dimension accurately.
4. Recognize an orthonormal collection.
5. Explain why orthonormal coordinates are convenient for measurement and reconstruction.

## Wed Sep 30 — Unit I synthesis

[Lecture 15 page](#)

**Reading:** Unit I synthesis (no new Boyd section)

**Note:** Same day includes Midterm 1 review ([Lecture 16](#) below).

By the end of class, students should be able to:

1. Connect distance, similarity, span, and independence.
2. Select an appropriate vector tool for a new problem.
3. Move between algebraic, geometric, computational, and contextual representations.
4. Diagnose and explain common errors.

## Wed Sep 30 — Midterm 1 review

[Lecture 16 page](#)

**Reading:** Unit I review

**Note:** Full review day.

By the end of class, students should be able to:

1. Retrieve major Unit I concepts without notes.
2. Solve mixed rather than chapter-isolated problems.
3. Explain reasoning collaboratively.
4. Identify gaps and make a targeted study plan.

## Fri Oct 2 — Midterm 1

[Lecture 17 page](#)

**Reading:** Midterm 1 (in class)

By the end of class, students should be able to:

1. Compute with vectors and linear functions.
2. Interpret vector operations in context.
3. Analyze distance, similarity, span, and independence.
4. Select and communicate an appropriate method.

## Week 7

## Mon Oct 5 — Matrices as data and structured objects

[Lecture 18 page](#)

**Reading:** Boyd 6.1–6.3

By the end of class, students should be able to:

1. Identify matrix dimensions, rows, columns, and entries.
2. Represent datasets, networks, images, or systems as matrices.
3. Perform matrix addition, scaling, and transposition.
4. Recognize zero, identity, diagonal, triangular, symmetric, and sparse matrices.
5. Explain one practical consequence of matrix structure.

## Wed Oct 7 — Matrix-vector multiplication

[Lecture 19 page](#)

**Reading:** Boyd 6.4

**Note:** Connect directly back to Week 1 linear combinations.

By the end of class, students should be able to:

1. Compute  $Ax$  by hand for a small matrix.
2. Interpret  $Ax$  as a linear combination of the columns of  $A$ .
3. Interpret  $A$  as an input-output transformation.
4. Explain how the dimensions of  $A$  determine the input and output spaces.

## Fri Oct 9 — Transforming space

[Lecture 20 page](#)

**Reading:** Boyd 7.1–7.2, [Lab 3](#), and Lachniet Ch. 2 (matrices and linear-algebra computations).

**Note:** Include a 3D or coordinate-system example.

By the end of class, students should be able to:

1. Predict the effect of scaling, reflection, shear, and rotation matrices.
2. Apply matrices to many 2D or 3D points at once.
3. Plot and compare original and transformed objects.
4. Adapt a short program that implements a transformation.
5. Interpret a matrix as a coordinate or physical-space transformation.

## Week 8

### Mon Oct 12 — Systems and $Ax = b$

[Lecture 21 page](#)

**Reading:** Boyd 8.3

By the end of class, students should be able to:

1. Translate equations or contextual constraints into  $Ax = b$ .
2. Interpret  $Ax = b$  as asking whether  $b$  belongs to the span of the columns of  $A$ .
3. Classify systems as square, underdetermined, or overdetermined.
4. Anticipate circumstances producing zero, one, or many solutions.

## Wed Oct 14 — Gaussian elimination

[Lecture 22 page](#)

**Reading:** Gaussian elimination (class notes) and Lachniet Ch. 2 (solving systems, RREF).

**Note:** Hand work first; software later.

By the end of class, students should be able to:

1. Perform elementary row operations.
2. Reduce a small augmented matrix to echelon form.
3. Solve a small system using back substitution.
4. Explain why row operations preserve the solution set.

## Fri Oct 16 — RREF, pivots, free variables, and software

[Lecture 23 page](#)

**Reading:** RREF and software solvers; Lachniet Ch. 2 (linear-system commands).

**Note:** For Python, provide an RREF helper or use SymPy rather than making novices implement RREF.

By the end of class, students should be able to:

1. Interpret reduced row echelon form.
2. Identify pivot and free variables.
3. Describe a solution set parametrically.
4. Distinguish inconsistent, unique, and infinite-solution cases.
5. Use software to solve or row-reduce a system and verify the answer using a residual.

## Week 9

### Mon Oct 19 — Linear dynamical systems

[Lecture 24 page](#)

**Reading:** Boyd 9.1–9.3

By the end of class, students should be able to:

1. Represent an evolving system as  $x_{t+1} = Ax_t$ .
2. Interpret the state vector and transition matrix.
3. Build a transition matrix from a flow description.
4. Compute and interpret the next few states.

### Wed Oct 21 — Simulation and long-run behavior

[Lecture 25 page](#)

**Reading:** Simulation of linear dynamics

**Note:** Do not explain the eigenvalue mechanism yet; let students form conjectures.

By the end of class, students should be able to:

1. Simulate many time steps computationally.
2. Plot state components through time.
3. Recognize stabilization, growth, decay, or oscillation.
4. Conduct a simple what-if simulation.

5. State limitations of a linear dynamical model.

## Fri Oct 23 — Predict a system through time

[Lecture 26 page](#)

**Reading:** Lab 4, Lachniet Ch. 2 (matrix operations), and Lachniet’s introductory programming/loops material.

**Note:** This creates the eigenvalue “mystery.”

By the end of class, students should be able to:

1. Implement a linear dynamical model.
2. Write or complete a short iterative calculation.
3. Validate the first steps by hand.
4. Compare results under altered parameters or initial states.
5. Record an unexplained long-run pattern for later investigation.

## Week 10

### Mon Oct 26 — Matrix multiplication

[Lecture 27 page](#)

**Reading:** Boyd 10.1

By the end of class, students should be able to:

1. Determine whether a product is defined and give its dimensions.
2. Compute a small matrix product.
3. Interpret  $ABx$  as a sequence of transformations.
4. Explain why matrix multiplication is generally not commutative.

### Wed Oct 28 — Composition and matrix powers

[Lecture 28 page](#)

**Reading:** Boyd 10.2–10.3

**Note:** Return to the dynamics lab from a new viewpoint.

By the end of class, students should be able to:

1. Combine multistage processes using a matrix product.
2. Connect  $A^k x$  to repeated dynamics.
3. Interpret selected entries of a matrix product or power.
4. Recognize how network and transition information accumulates through powers.

## Fri Oct 30 — Same mathematics, very different computational cost

[Lecture 29 page](#)

**Reading:** Boyd's  $ab^Tc$  example (micro-lab)

**Note:** Precursor to the capstone.

By the end of class, students should be able to:

1. Compare algebraically equivalent orders of computation.
2. Count intermediate values that must be stored.
3. Measure runtime for increasing problem sizes.
4. Explain why forming a large intermediate matrix may be wasteful.
5. Choose a computational order and justify it quantitatively.

## Week 11

### Mon Nov 2 — Inverses and solving

[Lecture 30 page](#)

**Reading:** Boyd 11.1–11.3

By the end of class, students should be able to:

1. Interpret an inverse as undoing a transformation.
2. Determine whether a simple matrix is invertible.
3. Connect invertibility to uniqueness of  $Ax = b$ .
4. Explain why solving  $Ax = b$  is generally preferable computationally to explicitly calculating  $A^{-1}$ .

### Wed Nov 4 — Rank, column space, null space, and basis connections

[Lecture 31 page](#)

**Reading:** Boyd plus selected portions of [Strang §3.5](#), especially rank, column space, null space, basis, and dimension.

**Note:** Not a full four-subspaces theory unit. Row space and left nullspace are enrichment, not quiz material.

By the end of class, students should be able to:

1. Interpret column space and null space through  $Ax = b$  and  $Ax = 0$ .
2. Determine rank from pivots or independent columns.
3. Find simple bases for column or null spaces.
4. Connect rank to consistency, independence, and dimension.
5. Use subspace, basis, dimension, rank, column space, and null space correctly.

### Fri Nov 6 — Determinants, geometry, and invertibility

[Lecture 32 page](#)

**Reading:** Determinants (class notes)

By the end of class, students should be able to:

1. Compute determinants of  $2 \times 2$  and simple triangular matrices.
2. Interpret determinant magnitude as area or volume scaling.
3. Connect determinant zero with lost dimension and noninvertibility.
4. Recognize the determinant of a triangular matrix from its diagonal.
5. Connect determinants to the coming characteristic equation.

## Week 12

### Mon Nov 9 — Midterm 2 review and synthesis

[Lecture 33 page](#)

**Reading:** Unit II review

**Note:** Full review day.

By the end of class, students should be able to:

1. Connect transformations, systems, dynamics, rank, inverses, and determinants.
2. Compare hand and computational solution methods.
3. Interpret software output instead of merely reporting it.
4. Solve a new application requiring more than one concept.

### Wed Nov 11 — Veterans Day — no class

### Fri Nov 13 — Midterm 2

[Lecture 34 page](#)

**Reading:** Midterm 2 (in class)

By the end of class, students should be able to:

1. Analyze matrices as transformations and models.
2. Solve and interpret systems.
3. Use matrix products, powers, rank, inverses, and determinants appropriately.
4. Interpret computational results in context.

## Week 13

### Mon Nov 16 — Eigenvectors as persistent directions

[Lecture 35 page](#)

**Reading:** Beginning of [Strang §6.1](#) —  $Ax = \lambda x$ , eigenvectors, and dynamics.

**Note:** Start with the unexplained behavior from Lab 4. The free PDF is only §6.1.

By the end of class, students should be able to:

1. Explain geometrically what makes  $v$  an eigenvector.
2. Verify a proposed eigenpair.
3. Find  $\lambda$  when an eigenvector is given.
4. Connect eigenvectors to repeated transformations and steady directions.
5. Recognize that a rotation may have no real eigenvector, motivating complex numbers.

## Wed Nov 18 — Characteristic equation, $2 \times 2$ eigenproblems, and complex numbers

[Lecture 36 page](#)

**Reading:** Strang §6.1, characteristic equation and the complex-eigenvalue ( $90^\circ$  rotation) example.

**Note:** Meets the explicit hand-calculation and complex-number CLOs. The same free §6.1 file is used Nov 16–20.

By the end of class, students should be able to:

1. Interpret  $a + bi$  algebraically and as a point/vector in the complex plane.
2. Perform the basic complex arithmetic needed for eigenvalue calculations.
3. Form and solve  $\det(A - \lambda I) = 0$  for a  $2 \times 2$  matrix.
4. Find corresponding eigenvectors.
5. Check real or complex eigenpairs by substitution.

## Fri Nov 20 — Why did the system behave that way?

[Lecture 37 page](#)

**Reading:** Strang §6.1, Lab 5, and Lachniet Ch. 5 (eigenvalues/eigenvectors).

**Note:** Normal modes are an application/demo, not a differential-equations derivation. Students then use software in Lab 5.

By the end of class, students should be able to:

1. Compute eigenvalues and eigenvectors of a general matrix with software.
2. Verify  $Av \approx \lambda v$  numerically.
3. Use dominant eigenbehavior to explain a simulated system.
4. Interpret complex eigenvalues as rotation/oscillation behavior.
5. Explore a symmetric-matrix or normal-mode example and observe orthogonal eigenvectors.

## Week 15

### Mon Nov 30 — When $Ax = b$ has no exact solution

[Lecture 38 page](#)

**Reading:** Boyd 12.1

By the end of class, students should be able to:

1. Recognize an inconsistent overdetermined system.
2. Formulate the least-squares objective  $\min_x \|Ax - b\|$ .
3. Interpret the fitted vector as the closest attainable vector.
4. Interpret the residual geometrically.
5. Distinguish an exact solution from a best approximate solution.

## Wed Dec 2 — Least squares, data fitting, and positive definiteness

[Lecture 39 page](#)

**Reading:** Boyd 12.2–12.4 and 13.1, class notes on symmetric and positive-definite matrices, and Lachniet Ch. 2 (least-squares / data-fitting exercises).

**Note:** Use a small real dataset. Positive definiteness is from Boyd and class notes, not later Strang sections. Software practice is [Lab 6](#) on Friday with Quiz 10.

By the end of class, students should be able to:

1. Compute a least-squares solution with Octave or Python.
2. Plot observed and fitted values.
3. Interpret residuals and an RMS error in context.
4. Check the residual's orthogonality to the columns of  $A$ .
5. Evaluate whether a fitted model is useful rather than merely reporting coefficients.
6. Recognize that  $A^T A$  is symmetric positive semidefinite, explain  $x^T A^T A x = \|Ax\|^2$ , and recognize that  $A^T A$  is positive definite when  $A$  has linearly independent columns.

## Fri Dec 4 — SVD, Lab 6, Quiz 10, and capstone launch

[Lecture 40 page](#) · [Lab 6](#) · [Quiz 10](#)

**Reading:** Selected slides from the [Strang/MIT SVD slides](#) (not all 17 pages), [Lab 6](#), capstone launch, Lachniet Ch. 5 (SVD) and Ch. 7 (image compression).

**Note:** Labs stay on Fridays — least-squares software is today alongside Quiz 10; SVD / capstone launch share the period.

By the end of class, students should be able to:

1. Interpret  $A = U\Sigma V^T$  conceptually.
2. Construct a rank- $k$  approximation computationally.
3. Compare singular-value decay for an image/frame and a frame-difference matrix.
4. Encode three frames either independently or as  $F_1, D_2, D_3$ .
5. Compare reconstruction error and storage for the two approaches.
6. Formulate a capstone question involving accuracy, storage, runtime, and resource use.

## Week 16

### Mon Dec 7 — Capstone studio

[Lecture 41 page](#)

**Reading:** Capstone (no new mathematics)

By the end of class, students should be able to:

1. Implement and validate a low-rank computational strategy.
2. Measure approximation error, storage, and runtime.
3. Separate one-time factorization cost from repeated-use cost.
4. Compare theoretical operation/storage counts with empirical measurements.
5. Debug and refine an analysis collaboratively while retaining individual understanding.

## Wed Dec 9 — Capstone presentations / gallery walk

[Lecture 42 page](#)

**Reading:** Capstone presentations

**Note:** Pair lightning talks or gallery walk.

By the end of class, students should be able to:

1. Explain the problem and why SVD or another linear-algebra tool was appropriate.
2. Present quantitative evidence about accuracy and cost.
3. Defend a choice of rank or representation.
4. Evaluate classmates' methods constructively.
5. Communicate the practical value and limitations of the result.

## Fri Dec 11 — Cumulative final review

[Lecture 43 page](#)

**Reading:** Cumulative review

**Note:** No routine quiz or new content.

By the end of class, students should be able to:

1. Connect vectors, systems, transformations, dynamics, eigenvectors, least squares, and SVD.
2. Select an appropriate method for a new situation.
3. Decide what should be done by hand and what should be delegated to software.
4. Interpret mathematical and computational results.
5. Create a targeted final-exam study plan.